You're building a PAN (Primary Account Number) generator for a card issuance system. When we issue a new virtual card, we need to generate a valid card number.
Build a CLI tool that generates a valid 16-digit PAN given:
bin — the Bank Identification Number (first 6 digits, e.g. "542759")

can you give me a breif draft of how this function will look like
count — how many PANs to generate
Requirements:

Each PAN must pass the Luhn algorithm
PANs must not collide with each other in the same batch
Output to stdout or a file

A simple approach: combine the **6-digit BIN + 9 unique random digits + 1 Luhn check digit**.

```
import argparse
import secrets
from pathlib import Path


def generate_pans(bin: str, count: int) -> list[str]:
    if len(bin) != 6 or not bin.isascii() or not bin.isdigit():
        raise ValueError("BIN must contain exactly 6 digits")
    if not 1 <= count <= 10**9:
        raise ValueError("Count must be between 1 and 1,000,000,000")

    # Sampling without replacement prevents collisions in this batch.
    suffixes = secrets.SystemRandom().sample(range(10**9), count)
    pans = []

    for suffix in suffixes:
        payload = f"{bin}{suffix:09d}"
        total = 0

        for i, char in enumerate(payload):
            digit = int(char)
            if i % 2 == 0:
                digit *= 2
                if digit > 9:
                    digit -= 9
            total += digit
```

```
            check_digit = (-total) % 10
            pans.append(f"{payload}{check_digit}")

        return pans


    if __name__ == "__main__":
        parser = argparse.ArgumentParser(description="Generate Luhn-valid PANs")
        parser.add_argument("bin", help="6-digit BIN")
        parser.add_argument("count", type=int)
        parser.add_argument("-o", "--output", help="Output file; defaults to stdo
        args = parser.parse_args()

        try:
            output = "\n".join(generate_pans(args.bin, args.count)) + "\n"
        except ValueError as error:
            parser.error(str(error))

        if args.output:
            Path(args.output).write_text(output, encoding="utf-8")
        else:
            print(output, end="")
```

Usage:

```
python test.py 542759 10
python test.py 542759 10 --output pans.txt
```

This guarantees uniqueness within the batch. Production issuance also needs a database uniqueness constraint across batches. Luhn validity alone does not make a PAN an issued card.

can you explain luhn algorhithm to me in this context

The **Luhn algorithm** calculates the final digit of a PAN so common typing errors can be detected.

Your 16-digit PAN has this structure:

```
542759 | 123456789 | ?
```

```
BIN    random      check digit
```

To calculate `?` :

1. Take the first 15 digits: `542759123456789` .
2. Starting from the **rightmost digit**, double every other digit. If the result exceeds 9, subtract 9.
3. Add all the resulting digits.
4. Choose a check digit that makes the total a multiple of 10.

For this example:

```
Original:    5  4  2  7  5  9  1  2  3  4  5  6  7  8  9
Transformed: 1  4  4  7  1  9  2  2  6  4  1  6  5  8  9
```

The sum is **69**, so the check digit is **1**:

```
69 + 1 = 70
PAN: 5427591234567891
```

In your function, this line calculates it:

```
check_digit = (-total) % 10
```

When **validating a complete PAN**, keep the final check digit unchanged and double every second digit moving left from it. The transformed sum must be divisible by 10.

Luhn checks the number's structure; it doesn't guarantee uniqueness or confirm that a card exists.

> can you add the code implemented to test.py under prestleystuffs folder?

Added the CLI to test.py.

```
python3 test.py 542759 10
python3 test.py 542759 10 --output pans.txt
```

Verified 1,000 PANs for Luhn validity and batch uniqueness, plus file output and invalid-input handling.

Edited test.py
+59 -0

if not 1 <= count <= 10**9:
raise ValueError("Count must be between 1 and 1,000,000,000")

can you explain to mw what this value means here

`10**9` means **10 to the power of 9**, which is **1,000,000,000**.

This condition checks that `count` is between **1 and 1 billion**, inclusive:

```
if not 1 <= count <= 10**9:
```

If it falls outside that range, the function raises a `ValueError`.

The upper limit comes from the PAN's structure:

```
6-digit BIN + 9 random digits + 1 check digit = 16 digits
```

Nine random digits allow **1 billion unique combinations**, from `000000000` to `999999999`. The check digit is calculated from the other digits, so it adds no extra combinations.

That's the theoretical limit per BIN; generating that many at once would require substantial memory.

542759 | 123456789 | ?

```
542759 | 123456789 | 1
  BIN     suffix    Luhn check digit
6 digits  9 digits  1 digit
```

For this example, the Luhn calculation gives a check digit of **1**, making the complete 16-digit PAN:

```
5427591234567891
```

suffixes = secrets.SystemRandom().sample(range(10**9), count)

what does this mean

This line picks `count` **unique random numbers** to use as the nine-digit suffixes.

```
suffixes = secrets.SystemRandom().sample(range(10**9), count)
```

Breaking it down:

- `secrets.SystemRandom()` uses the operating system's secure randomness.
- `range(10**9)` represents numbers from `0` to `999,999,999`.
- `.sample(..., count)` picks that many numbers **without repeats**.

For example, if `count = 3`, the result could be:

```
suffixes = [123456789, 42, 987654321]
```

Later, this formatting ensures each suffix has exactly nine digits:

```
f"{suffix:09d}"
```

So `42` becomes `"000000042"`. Each unique suffix produces a unique PAN for the given BIN.

for suffix in suffixes:
payload = f"{bin}{suffix:09d}"
total = 0

```
        for i, char in enumerate(payload):
            digit = int(char)
            # In a 16-digit PAN, double positions 0, 2, ...,
            if i % 2 == 0:
                digit *= 2
                if digit > 9:
                    digit -= 9
            total += digit

        check_digit = (-total) % 10
        pans.append(f"{payload}{check_digit}")

    return pans
```

is this part of the code to verify the checksum numbers for the luhn alogrhithm?

Yes, it uses the Luhn algorithm, but this code **generates the check digit** rather than verifying an existing one.

Here's what it does:

1. **Build the first 15 digits:**

```
payload = f"{bin}{suffix:09d}"
```

   Combines the 6-digit BIN with the 9-digit suffix.
2. **Calculate the Luhn total:**

```
if i % 2 == 0:
    digit *= 2
    if digit > 9:
```

```
            digit -= 9
    total += digit
```

Doubles digits at positions `0, 2, …, 14`, subtracts 9 when needed, and adds the transformed values.

3. **Calculate the final check digit:**

```
check_digit = (-total) % 10
```

Chooses the digit that makes the total divisible by 10. For example, if `total = 69`, the check digit is `1`.

4. **Save the complete 16-digit PAN:**

```
pans.append(f"{payload}{check_digit}")
```

To **verify** a complete PAN, you would perform the Luhn transformation including its existing check digit and check whether `total % 10 == 0`.

> PANs must not collide with each other in the same batch
>
> this is paert of the requirement of the function. can you check thoroughly to make sure that this requirement is already implemented in the function? if not, provide me with the draft of this requirement before implementing it into the test.py file

Yes—the requirement is already implemented in test.py (line 16). No changes are needed.

This line guarantees unique suffixes within each batch:

```
suffixes = secrets.SystemRandom().sample(range(10**9), count)
```

`.sample()` selects numbers **without replacement**, so the same suffix cannot appear twice.

Each PAN consists of:

```
Same BIN + unique 9-digit suffix + calculated check digit
```

Because the suffixes differ, the PANs must differ—even if some have the same check digit. Padding suffixes with zeros preserves their uniqueness.

I verified:

- **303,003 PANs across 9 batches:** no duplicates and correct output counts.
- Every PAN had 16 digits, the requested BIN, and a valid Luhn checksum.
- Boundary suffixes (`0` , `1` , `999999999`) stayed distinct after formatting.
- Counts outside the permitted range were rejected.

This guarantees uniqueness **within one function call**. Separate calls could generate overlapping PANs, which is outside the stated batch requirement.

> Each PAN must pass the Luhn algorithm
>
> this is the first requirement of the function. the requirement is already in the code based on the checksum algorhithm from line 18 onwards right?

Yes, that requirement is already implemented in the loop from around line 19 onward.

The code:

1. Combines the BIN and suffix into 15 digits.
2. Applies the Luhn transformation and sums the results.
3. Calculates the check digit:

```
check_digit = (-total) % 10
```

4. Appends it to produce a valid 16-digit PAN:

```
pans.append(f"{payload}{check_digit}")
```

The calculated check digit makes the Luhn total divisible by 10, so **every generated PAN passes Luhn validation**. The independent checks I ran also confirmed this.

Output to stdout or a file

is the output correct as per the requirement?

Yes. The `main()` function supports both options:

Without `--output`, PANs are printed to stdout, one per line:

```
python3 test.py 542759 10
```

```
print(output, end="")
```

With `--output` or `-o`, PANs are written to the specified file, one per line:

```
python3 test.py 542759 10 --output pans.txt
```

```
Path(args.output).write_text(output, encoding="utf-8")
```

Both output modes were tested successfully. Note that writing to an existing file overwrites its contents.

can you rerun the whole code thoroughly and make sure it adheres to these 3 requirements:

Each PAN must pass the Luhn algorithm
PANs must not collide with each other in the same batch
Output to stdout or a file

Yes—test.py passes all three requirements.

I ran **36 CLI batches generating 909,099 PANs**:

Requirement	Result
Each PAN passes Luhn	All passed an independent validator
No collisions within a batch	No duplicates in any batch
Output to stdout or a file	Both worked, including `-o` and `--output`

I also checked suffix boundaries, invalid arguments, and file-write errors. No code changes were needed.

Uniqueness is guaranteed within each batch; separate batches may overlap.

> for the sample batches you carried out, is is guaranteed to pick a unique set

Yes—within each batch, uniqueness is guaranteed, because:

```
secrets.SystemRandom().sample(range(10**9), count)
```

`sample()` selects **without replacement**, so every suffix in that batch is different. Combining those suffixes with the BIN produces distinct PANs.

The tests confirmed this behavior; the guarantee comes from how `sample()` works, rather than from the test results alone.

Across separate batches, uniqueness is not guaranteed. A later call could select a suffix used in an earlier call.